

Amazon EBS

Study Notes & Task Guide

Understanding EBS: Concepts & Console Walkthroughs

Contents

1. What is Amazon EBS?
2. Storage Types: Block vs Object
3. Key Concepts — IOPS, Throughput, Block Size, Queue Depth
4. Volume Types — Standard (gp2) vs Provisioned IOPS (io1)
5. Core Features of EBS
6. Task: Create a Volume [Console]
7. Task: Attach a Volume to an EC2 Instance [Console]
8. Task: Mount a Volume on Linux [CLI — inside EC2]
9. Task: Detach and Delete a Volume [Console]
10. Task: Snapshots — Create, Restore, Copy [Console]
11. Task: Resize / Modify a Volume [CLI — after Console modify]
12. Task: Enable Encryption [Console]
13. Performance Tips
14. Quick Reference & Cheat Sheet

Contents

- 1. What is Amazon EBS?**
- 2. Storage Types — Block vs Object**
- 3. Key Concepts**
 - 3.1 IOPS and Throughput
 - 3.2 Block Size
 - 3.3 Queue Depth
- 4. Volume Types — gp2 vs io1**
- 5. Core Features of EBS**
- 6. Task: Create a Volume [Console]**
- 7. Task: Attach a Volume [Console]**
- 8. Task: Mount a Volume on Linux [CLI]**
- 9. Task: Detach and Delete a Volume [Console]**
- 10. Task: Snapshots — Create, Restore, Copy [Console]**
- 11. Task: Resize / Modify a Volume [CLI]**
- 12. Task: Enable Encryption [Console]**
- 13. Performance Tips**
- 14. Quick Reference & Cheat Sheet**

1. What is Amazon EBS?

Amazon Elastic Block Store (EBS) is a block-level storage service for EC2 instances. It behaves like a virtual hard drive attached to your cloud server — you can create a file system on it, run a database, store application data, and so on.

The key difference from a physical hard drive: EBS volumes exist independently of the EC2 instance. Stopping or terminating an instance leaves the volume and all its data intact. You can then attach it to a different instance.

Tip:

Think of EBS as an external USB drive for your cloud server. The server (EC2) and the drive (EBS) are separate. You can unplug the drive, move it, or keep it after the server is gone.

Persistent	Data survives instance stop, restart, and termination.
Independent	Volumes exist separately from instances. Delete the server — keep the volume.
Attachable	Detach from one instance and attach to another in the same Availability Zone.
Snapshotable	Take point-in-time backups (snapshots) at any time with one click.
Scalable	Increase size or change performance type without stopping the server.
Encrypted	Enable AES-256 encryption with a single checkbox.

2. Storage Types — Block vs Object

AWS offers several storage services. The most important distinction for beginners is between Block Storage (EBS) and Object Storage (S3). Picking the wrong one is a very common mistake.

	Block Storage (EBS)	Object Storage (S3 / Glacier)
Analogy	Hard drive plugged into your computer	Filing cabinet accessed over the internet
Use case	OS, databases, application files, logs	Images, videos, PDFs, backups, static files
Access	Read/write via a file system	Upload / download whole files via URL
Shared access	One server at a time only	Many users and services simultaneously
Performance	Very fast, low latency	Slower, optimised for large transfers
Cost model	Per GB provisioned per month	Per GB stored + per API request

Important:

If your application needs a file system or runs a database — use EBS. If you need to store files that users download or that many services access — use S3.

3. Key Concepts

3.1 IOPS and Throughput

These two metrics describe storage performance. They sound similar but measure different things.

Term	What it measures	Unit
IOPS	How many read or write operations the disk handles per second	operations/sec
Throughput	How much data moves through the connection per second	MB/s

When each one matters:

- IOPS matters for databases and apps doing many small random reads/writes.
- Throughput matters for big sequential transfers — video, analytics, large files.

Note:

Networks are in Mbit/s; disks are in MB/s. Divide by 8 to convert: 1,000 Mbit/s = 125 MB/s. At 1,000 Mbit/s: roughly 7,500 ops of 16 KB fit, or 30,000 ops of 4 KB. Smaller blocks = more IOPS possible on the same connection.

3.2 Block Size

A block is the smallest chunk of data EBS reads or writes in one operation. Smaller blocks allow more IOPS; larger blocks move more data per trip.

Typical DB block	4 KB — default for MySQL, PostgreSQL
PIOPS limit	16 KB per I/O on io1 volumes
Throughput formula	Throughput = IOPS x Block Size (e.g. 2,000 x 16 KB = 32 MB/s)
Larger blocks	32 KB blocks on a 2,000 PIOPS volume halves IOPS to 1,000 but keeps throughput at 32 MB/s

3.3 Queue Depth

Queue depth is the number of I/O requests waiting in line at any moment. The disk works fastest when there is always a queue. If the queue is empty, the disk sits idle and you waste the IOPS you paid for.

Rule	Keep 1 pending I/O for every 200 provisioned IOPS
2,000 PIOPS	Maintain at least 10 requests in flight at all times
500 PIOPS	Maintain at least 3 requests in flight at all times

Auto-managed	Most databases (MySQL, PostgreSQL, MongoDB) handle this automatically
---------------------	---

4. Volume Types — gp2 vs io1

	Standard — gp2	Provisioned IOPS — io1
Average IOPS	~100 IOPS, occasional bursts	Up to 4,000 IOPS — guaranteed
Consistency	Best-effort, not guaranteed	Within 10% of provisioned IOPS, 99.9% of the year
Best for	Boot volumes, websites, dev/test, logs	MySQL, PostgreSQL, Oracle, MongoDB, Exchange
Max size	Up to 1 TB	Up to 1 TB
IOPS guarantee	None	Delivers within 10% of provisioned value, 99.9% uptime SLA
Cost	Lower	Higher — you pay per provisioned IOPS

When to use gp2

- Boot / OS volumes for EC2 instances
- Development and test environments
- Websites and blogs with moderate traffic
- Log processing or analytics with sequential reads
- Any workload averaging under 100 IOPS

When to use io1 (PIOPS)

- Production relational databases: MySQL, PostgreSQL, Oracle, SQL Server
- NoSQL databases: MongoDB, Cassandra
- Enterprise applications: SAP, Microsoft Exchange
- Any workload where inconsistent I/O causes errors or slowdowns

Note:

PIOPS SLA: a 200 GB io1 volume with 1,000 IOPS provisioned will deliver at least 900 IOPS (within 10%) for 99.9% of the year — roughly only 8.7 hours per year could fall below the guarantee.

5. Core Features of EBS

High-performance file system	Mount EBS as a drive, format with ext4 / NTFS / XFS, and use it like a local disk.
Flexible size	1 GB to 1 TB. Expand an existing volume without stopping the server or losing data.
Private and secure	Private to your AWS account. Only the attached EC2 instance can read or write data.
Replication within AZ	AWS automatically replicates every volume within its Availability Zone. If a physical disk fails, the replica takes over transparently.
Snapshots	Point-in-time backups stored in S3. First snapshot is a full copy; subsequent ones are incremental (only changed blocks are uploaded).
CloudWatch monitoring	EBS sends metrics every minute: IOPS, throughput, latency, queue depth. Set alarms to alert you when performance degrades.

6. Task: Create a Volume [Console]

Console path:

AWS Console > EC2 > Elastic Block Store > Volumes > Create Volume

1 Sign in to the AWS Console

Go to console.aws.amazon.com. Enter your email, password, and MFA code if prompted.

Note: Check the region selector in the top-right corner before doing anything. Your volume will be created in whichever region is currently selected.

2 Go to EC2

In the top search bar type EC2 and click the EC2 result. This opens the EC2 Dashboard.

3 Open the Volumes page

In the left sidebar, under the Elastic Block Store heading, click Volumes. You will see a list of any existing volumes in your account.

4 Click Create Volume

Click the Create Volume button in the top-right area of the page. The Create Volume form opens.

5 Choose the Volume Type

In the Volume Type dropdown select one of the following: - gp2 (General Purpose SSD): suitable for most beginner workloads, ~100 IOPS average - io1 (Provisioned IOPS SSD): for production databases needing guaranteed IOPS - st1 (Throughput Optimized HDD): large sequential workloads such as big data - sc1 (Cold HDD): cheapest option, for data accessed rarely Select gp2 if you are unsure. You can change type later without downtime.

Note: Start with gp2 and only upgrade to io1 when your workload clearly needs it.

6 Set the Size

Type the volume size in GiB in the Size field. Minimum is 1 GiB; maximum is 1,024 GiB (1 TiB). Example: enter 100 for a 100 GB volume.

Note: You can increase the size of a volume later without downtime, but you cannot decrease it.

7 Set IOPS (io1 volumes only)

This field only appears when you select io1 as the volume type. Enter the number of IOPS your application requires. Minimum: 100. Maximum: 4,000. Example: enter 2000 for a standard production database.

Note: Provision 20% more IOPS than your expected peak — it is easy to reduce later but harder to recover from an under-provisioned database under load.

8 Choose the Availability Zone

Click the Availability Zone dropdown and select the AZ where your EC2 instance lives. To check your instance's AZ: open EC2 > Instances, find your instance, and look at the Availability Zone column. Example: if the instance is in us-east-1a, select us-east-1a here.

Note: This is the most common setup mistake. An EBS volume can only be attached to an instance in the same AZ. Wrong AZ = cannot attach.

9 Snapshot ID (leave blank for a new empty volume)

Leave this field empty if you want a fresh volume with no data. If you are restoring a backup, type or search for the snapshot ID here. Restoring from snapshots is covered in Section 10.

10 Enable Encryption

Tick the Encrypt this volume checkbox. In the KMS Key dropdown, leave the default (aws/ebs) unless you have created a custom key. Encryption covers data at rest on the volume, data in transit between the volume and the instance, and all future snapshots.

Note: Always encrypt production volumes. The performance overhead is negligible. Many compliance standards (PCI-DSS, HIPAA) require encryption at rest.

11 Add Tags

Click Add Tag. Add at least these three tags so you can identify the volume later: - Key: Name | Value: my-database-volume - Key: Environment | Value: production - Key: Owner | Value: your-team-name

Note: Without tags, volumes become impossible to identify once you have more than a few. Tags also appear in your AWS Cost Explorer billing reports.

12 Click Create Volume

Review every field one more time, then click the Create Volume button. A green success banner appears showing your new Volume ID (format: vol-0xxxxxxxxxxxxxxxxx). Write it down.

13 Confirm the volume reaches Available status

You are returned to the Volumes list. Find your new volume. The State column shows creating briefly, then changes to available. Once it shows available the volume is ready to attach. This normally takes under 60 seconds.

Note: Do not try to attach a volume before it reaches the available state.

7. Task: Attach a Volume to an EC2 Instance [Console]

Creating a volume is not enough — you must attach it to an EC2 instance before the server can see or use it.

Important:

The EBS volume and the EC2 instance must be in the same Availability Zone. If your instance is in us-east-1a the volume must also be in us-east-1a. You cannot attach across AZs.

Console path:

EC2 > Elastic Block Store > Volumes > [select volume] > Actions > Attach Volume

1 Go to Volumes and find your volume

In the left sidebar click Volumes under Elastic Block Store. Locate the volume you created in Section 6. Its State should show available.

2 Select the volume

Click the checkbox to the left of the volume row to select it. The row highlights and the Actions menu becomes active.

3 Open Actions and choose Attach Volume

Click the Actions dropdown button near the top of the page. Click Attach Volume in the dropdown list. A dialog box opens.

4 Select your EC2 instance

Click inside the Instance field and start typing your instance ID (starts with i-) or the instance name you gave it. A dropdown list will appear showing matching instances in the same AZ. Click your instance to select it.

Note: Only instances in the same Availability Zone appear in the list. If your instance is missing, the AZs do not match — go back and check.

5 Confirm or change the Device Name

The Device name field shows a suggested path such as /dev/sdf. Accept this default. Inside your Linux instance this will appear as /dev/xvdf or /dev/nvme1n1 depending on the instance type. You do not need to change this unless you have a specific reason to.

6**Click Attach Volume**

Click the Attach Volume button in the dialog. The dialog closes and you return to the Volumes list. Within a few seconds the State column changes from available to in-use and the Attached Instances column shows your instance ID.

Note: If State stays at available after 30 seconds, refresh the page and check that you selected the correct instance.

8. Task: Mount a Volume on Linux [CLI — inside EC2]

After attaching, the volume is visible to the operating system as a block device but is not yet accessible as a folder. You must format it (new volumes only) and mount it. All commands below are run inside your EC2 instance via SSH.

Important:

Only format a volume with `mkfs` on a brand new, empty volume. Formatting permanently erases all data. If remounting an existing volume that already has data, skip the format step.

Connect to the instance first

SSH

```
ssh -i /path/to/key.pem ec2-user@
```

```
# Ubuntu instances: use ubuntu instead of ec2-user
```

```
ssh -i /path/to/key.pem ubuntu@
```

Step 1 — List block devices

lsblk

```
lsblk
```

```
# Example output:
```

```
# NAME SIZE TYPE MOUNTPOINT
```

```
# xvda 8G disk / <- root volume (already mounted)
```

```
# xvd f 100G disk <- your new volume (no MOUNTPOINT = not mounted)
```

Note:

Your new volume appears without a MOUNTPOINT value. That confirms it is attached but not yet mounted.

Step 2 — Check whether the volume already has a file system

Check filesystem

```
sudo file -s /dev/xvdf
```

```
# If output is: /dev/xvdf: data
```

```
# -> volume is empty, proceed to Step 3 to format it.
```

```
# If output is: /dev/xvdf: Linux rev 1.0 ext4 filesystem ...
```

```
# -> volume already has data, SKIP Step 3 and go straight to Step 4.
```

Step 3 — Format the volume (new empty volumes only)

Format

```
# Format with ext4 (most common for Linux)
```

```
sudo mkfs -t ext4 /dev/xvdf
```

```
# Alternative: xfs (better for large files and databases)
```

```
sudo mkfs -t xfs /dev/xvdf
```

Note:

ext4 is the safest choice for beginners. Use xfs for large-file workloads.

Step 4 — Create a mount directory

Create mount point

```
sudo mkdir /data
```

```
# Other common names: /database /mnt/storage /var/lib/mysql
```

Step 5 — Mount the volume

Mount

```
sudo mount /dev/xvdf /data

# Verify it worked:

df -h

# You should see a line like:

# /dev/xvdf 99G 60M 94G 1% /data
```

Step 6 — Make the mount permanent across reboots

fstab — permanent mount

```
# 1. Get the UUID of the volume (UUID is stable; device name can change)

sudo blkid /dev/xvdf

# Output example: /dev/xvdf: UUID='a1b2c3d4-e5f6-...' TYPE='ext4'

# 2. Open /etc/fstab

sudo nano /etc/fstab

# 3. Add this line at the bottom (replace the UUID with yours):

UUID=a1b2c3d4-e5f6-... /data ext4 defaults,nofail 0 2

# 4. Save and exit: Ctrl+O then Enter then Ctrl+X

# 5. Test the fstab entry without rebooting:

sudo mount -a

# No output = success. An error means there is a typo in the UUID or path.
```

Tip:

The nofail option means the server boots normally even if the volume cannot be mounted — useful if you ever detach the volume without updating fstab.

Step 7 — Set permissions

Permissions

```
# Give ownership to your application user (e.g. ec2-user)
```

```
sudo chown ec2-user:ec2-user /data
```

```
# Verify
```

```
ls -la / | grep data
```


9. Task: Detach and Delete a Volume [Console]

Detaching a volume removes it from the instance but keeps all data on the volume. Deleting a volume permanently destroys the volume and all data on it.

Important:

Always unmount the volume inside the instance before detaching from the console. Detaching a mounted volume can corrupt your data.

Step 1 — Unmount inside the instance (SSH into your EC2 first)

Unmount

```
sudo umount /data
```

```
# If you get 'target is busy', find what is using it:
```

```
lsof /data
```

```
# Stop the process shown, then retry umount.
```

Step 2 — Detach via Console

Console path:

EC2 > Volumes > [select volume] > Actions > Detach Volume

1 Find the volume in the Volumes list

In the left sidebar click Volumes. Find your volume — State should show in-use.

2 Select the volume and open Actions

Click the checkbox next to the volume. Click the Actions dropdown and choose Detach Volume.

3 Confirm the detach

A confirmation dialog appears. Click Detach. The State column changes from in-use to available within a few seconds. The volume is now detached and can be safely deleted or reattached.

Step 3 — Delete via Console (only if you no longer need the volume)

Console path:

EC2 > Volumes > [select volume] > Actions > Delete Volume

1 Select the volume

The volume must be in the available state before you can delete it. If it still shows in-use, complete Step 2 — Detach first.

2 Click Actions > Delete Volume

Click the Actions dropdown and choose Delete Volume. A confirmation dialog warns you that this action is permanent.

3 Confirm the deletion

Read the warning carefully, then confirm. The volume disappears from the list. There is no undo and no recycle bin.

Note: Take a snapshot before deleting any volume that contains important data. Snapshots are covered in Section 10.

10. Task: Snapshots — Create, Restore, Copy [Console]

A snapshot is a point-in-time backup of an EBS volume stored in Amazon S3. The first snapshot copies all used data; subsequent snapshots are incremental — only blocks that changed since the last snapshot are uploaded.

Where stored	Amazon S3 — AWS manages the bucket, you do not interact with it directly
First snapshot	Full copy of all used blocks on the volume
Later snapshots	Incremental — only changed blocks
Volume in use?	Yes, you can snapshot an attached, running volume
Database tip	For consistency, freeze writes or snapshot a read replica
Cross-region	Snapshots can be copied to another region for disaster recovery

10.1 Create a Snapshot

Console path:

EC2 > Elastic Block Store > Volumes > [select volume] > Actions > Create Snapshot

1 Select the volume

In the Volumes list, click the checkbox next to the volume you want to back up.

2 Open Actions and click Create Snapshot

Click the Actions dropdown and choose Create Snapshot. A dialog box opens with a Description field and optional Tags.

3 Write a clear description

Enter a description that includes the date and the reason. Example: prod-db backup 2024-01-15 before v2.0 deployment Add tags: Name = prod-db-backup, Date = 2024-01-15

Note: Good descriptions are essential. Without them you cannot tell which snapshot to restore from when you have dozens of them.

4 Click Create Snapshot

Click the Create Snapshot button. A success banner shows the new Snapshot ID (format: snap-0xxxxxxxxxxxxxxxxx). Note it down.

5 Monitor the progress

In the left sidebar click Snapshots under Elastic Block Store. Find your snapshot. Status shows pending while it uploads, then changes to completed when done. Small volumes complete in seconds; large volumes may take several minutes.

10.2 Restore a Volume from a Snapshot

Console path:

EC2 > Elastic Block Store > Snapshots > [select snapshot] > Actions > Create Volume from Snapshot

1 Open Snapshots and select the one to restore

Click Snapshots in the left sidebar. Find the snapshot you want. Click its checkbox to select it.

2 Click Actions > Create Volume from Snapshot

The Create Volume form opens. The Snapshot ID field is already filled with the selected snapshot.

3 Set the Availability Zone

Set the AZ to match the EC2 instance you plan to attach this restored volume to. You can also increase the volume size beyond the original snapshot size if needed. Click Create Volume.

Note: Restoring creates a brand new volume. The original snapshot is not affected.

4 Attach the restored volume

Follow Section 7 to attach the new volume to your instance. If you are replacing an existing volume, detach the old one first (Section 9).

10.3 Copy a Snapshot to Another Region

Console path:

EC2 > Snapshots > [select snapshot] > Actions > Copy Snapshot

Copying a snapshot to another region protects your backup against a regional AWS outage and lets you restore infrastructure in a different region.

1 Select the snapshot and click Actions > Copy Snapshot

In the Snapshots list, checkbox the snapshot. Click Actions > Copy Snapshot. A dialog opens.

2 Select the Destination Region

Choose the target region from the Destination Region dropdown. Example: eu-west-1 (Ireland) if your primary region is us-east-1 (N. Virginia). Optionally tick Encrypt this snapshot if the source is unencrypted.

3 Add a description and click Copy Snapshot

Add a description such as DR copy us-east-1 2024-01-15. Click Copy Snapshot. To verify the copy, switch to the destination region using the region selector in the top-right corner and open Snapshots.

11. Task: Resize / Modify a Volume [CLI]

You can modify an EBS volume while it is attached and in use — no server restart needed. You can increase the size, change the volume type, or change IOPS. You cannot decrease the size.

Important:

After resizing via the CLI or Console, AWS resizes the underlying block device but does NOT extend the file system. You must extend the file system manually inside the instance — otherwise the OS still sees the old size.

Modify the volume

Modify via CLI

```
# Increase size from 100 GB to 200 GB
```

```
aws ec2 modify-volume \
```

```
--volume-id vol-0abc123def456789a \
```

```
--size 200
```

```
# Change from gp2 to io1 with 2,000 IOPS
```

```
aws ec2 modify-volume \
```

```
--volume-id vol-0abc123def456789a \
```

```
--volume-type io1 \
```

```
--iops 2000
```

```
# Check modification progress
```

```
aws ec2 describe-volumes-modifications \
```

```
--volume-ids vol-0abc123def456789a
```

```
# Look for: 'ModificationState': 'completed'
```

Extend the file system inside the instance

After the modification reaches completed state, SSH into your instance and run the appropriate commands for your file system type.

Extend file system after resize

```
# --- ext4 file system ---
```

```
# Step 1: extend the partition (required if the volume has a partition table)
```

```
sudo growpart /dev/xvdf 1
```

```
# Step 2: extend the ext4 file system
```

```
sudo resize2fs /dev/xvdf
```

```
# Verify the new size
```

```
df -h
```

```
# --- xfs file system ---
```

```
# Single command (run while the volume is mounted at /data)
```

```
sudo xfs_growfs /data
```

```
# Verify
```

```
df -h
```

Tip:

You do not need to unmount the volume to extend the file system. Both `resize2fs` and `xfs_growfs` work on live, mounted volumes.

12. Task: Enable Encryption [Console]

EBS encryption uses AES-256. It protects data at rest on the volume, data in transit between the volume and the EC2 instance, and all snapshots created from the volume. Your application needs no code changes.

Tip:

The easiest approach: enable encryption at creation time (Section 6, Step 10). Simply tick the Encrypt this volume checkbox before clicking Create Volume.

Encrypting an Existing Unencrypted Volume

EBS does not let you enable encryption on an existing unencrypted volume directly. Use this five-step process instead.

1 Create a snapshot of the unencrypted volume

Go to EC2 > Volumes. Select the unencrypted volume. Click Actions > Create Snapshot. Add a description such as pre-encrypt snapshot. Wait for Status to show completed in the Snapshots page.

Note: This snapshot is unencrypted. You will encrypt it in the next step.

2 Copy the snapshot with encryption enabled

Go to EC2 > Snapshots. Select the snapshot you just created. Click Actions > Copy Snapshot. In the Copy Snapshot dialog: - Set Destination Region to the same region (unless you want a cross-region copy) - Tick the Encrypt this snapshot checkbox - Leave KMS Key as the default (aws/ebs) - Add a description: encrypted copy Click Copy Snapshot. Wait for the new snapshot Status to show completed.

Note: When done, you will have two snapshots: one unencrypted (original) and one encrypted (copy).

3 Create a new encrypted volume from the encrypted snapshot

In the Snapshots list, select the encrypted snapshot (the copy, not the original). Click Actions > Create Volume from Snapshot. Set the Availability Zone to match your EC2 instance. Confirm that the Encryption field shows Encrypted. Click Create Volume and wait for it to reach the available state.

4 Swap the old volume for the new encrypted one

1. Stop your EC2 instance: EC2 > Instances > select instance > Instance State > Stop 2. Detach the old unencrypted volume: EC2 > Volumes > select old volume > Actions > Detach Volume 3. Attach the new encrypted volume: EC2 > Volumes > select new volume > Actions > Attach Volume Use the same device path as the old volume (e.g. /dev/sda1 for the root volume) 4. Start the instance: EC2 > Instances > select instance > Instance State > Start 5. Verify the instance boots correctly and your data is intact.

Note: For a root volume swap, the instance must be stopped. Data volumes can be swapped while the instance is running if you unmount first.

5 Delete old unencrypted assets

Once you have confirmed the encrypted volume is working correctly: - Delete the old unencrypted volume: EC2 > Volumes > Actions > Delete Volume - Delete the unencrypted snapshot: EC2 > Snapshots > Actions > Delete Snapshot Keep the encrypted snapshot as your ongoing backup.

Note: Do not delete the old volume until you are 100% sure the encrypted one works. There is no undo.

13. Performance Tips

Use EBS-Optimized Instances

By default EC2 instances share network bandwidth between regular traffic and EBS I/O. An EBS-Optimized instance gives EBS its own dedicated network connection so storage performance is never affected by application network traffic.

- Always use EBS-Optimized instances with io1 (PIOPS) volumes
- Most modern instance families (M5, C5, R5, T3) are EBS-Optimized by default
- Older instance types require enabling EBS-Optimized manually at launch

Pre-Warm New PIOPS Volumes

New io1 volumes deliver 5–50% fewer IOPS the first time each block is accessed. Pre-warm before production use by writing zeros to every block. Run this inside the EC2 instance before mounting the volume for production.

Pre-warm

```
# WARNING: only on NEW, EMPTY volumes – this erases all data
```

```
sudo dd if=/dev/zero of=/dev/xvdf bs=1M status=progress
```

```
# Or use fio for more controlled pre-warming:
```

```
sudo fio --filename=/dev/xvdf --rw=write --bs=128k \
```

```
--iodepth=32 --ioengine=libaio --direct=1 --name=init
```

Tip:

Pre-warming a 500 GB volume can take 30–90 minutes. Volumes restored from snapshots do NOT need pre-warming.

Avoid Throughput Saturation

Attaching too many high-IOPS volumes to a single instance can saturate its network bandwidth and slow down all volumes.

Formula	Max safe volumes = Instance bandwidth / (IOPS per vol x block size)
M3.2XLarge (1 Gbps)	125 MB/s. At 500 IOPS x 16 KB per vol = 8 MB/s -> max ~15 volumes

CC2 (2 Gbps)	250 MB/s. At 2,000 IOPS x 16 KB per vol = 32 MB/s -> max ~7 volumes
---------------------	---

RAID

RAID 0 (Striping)	Splits data across volumes. Combined IOPS and throughput. No redundancy — one volume failure loses all data. Good when snapshots are your backup strategy.
RAID 10 (Stripe+Mirror)	Half the volumes mirror the other half. Combined performance + fault tolerance. Recommended for production databases. MongoDB, PostgreSQL, and MySQL all benefit from this.

Snapshot Schedule Best Practices

- Take daily snapshots at minimum — automate with AWS Data Lifecycle Manager (DLM)
- Snapshot a read replica rather than the primary to avoid I/O impact
- Copy snapshots to another region for disaster recovery
- Delete old snapshots you no longer need — each one costs S3 storage

CloudWatch Alarms to Configure

VolumeReadOps / WriteOps	Alert when IOPS consumed approaches the provisioned limit
VolumeQueueLength	Alert when queue length consistently exceeds the recommended depth
VolumeThroughputPercentage	Alert when approaching 100% — indicates saturation

14. Quick Reference & Cheat Sheet

Glossary

EBS	Elastic Block Store — AWS persistent block storage service
EC2	Elastic Compute Cloud — AWS virtual server
gp2	General Purpose SSD — ~100 IOPS average, for general workloads
io1	Provisioned IOPS SSD — guaranteed IOPS, for databases
IOPS	Input/Output Operations Per Second
Throughput	Data transferred per second (MB/s)
Block size	Smallest unit of data in one I/O operation
Queue depth	Number of I/O requests waiting to be processed
Snapshot	Incremental point-in-time backup stored in S3
AZ	Availability Zone — isolated data centre within a region
RAID 0	Striping — max performance, no redundancy
RAID 10	Stripe + Mirror — performance + redundancy
EBS-Optimized	EC2 instance with dedicated bandwidth for EBS traffic
Pre-warming	Writing to all blocks on a new volume before first use
KMS	Key Management Service — manages encryption keys
DLM	Data Lifecycle Manager — automates snapshot scheduling

Console Navigation Paths

Create a volume	EC2 > Elastic Block Store > Volumes > Create Volume
Attach a volume	EC2 > Volumes > [select] > Actions > Attach Volume
Detach a volume	EC2 > Volumes > [select] > Actions > Detach Volume
Delete a volume	EC2 > Volumes > [select] > Actions > Delete Volume
Create a snapshot	EC2 > Volumes > [select] > Actions > Create Snapshot
View snapshots	EC2 > Elastic Block Store > Snapshots
Restore from snapshot	EC2 > Snapshots > [select] > Actions > Create Volume from Snapshot

Copy a snapshot	EC2 > Snapshots > [select] > Actions > Copy Snapshot
Delete a snapshot	EC2 > Snapshots > [select] > Actions > Delete Snapshot
Modify / resize volume	EC2 > Volumes > [select] > Actions > Modify Volume
View volume metrics	EC2 > Volumes > [select] > Monitoring tab
Encrypt existing vol	Snapshot it > Copy snapshot (tick Encrypt) > Create vol from encrypted copy

Linux Commands — Inside the Instance (Sections 8 and 11)

```
lsblk # list block devices and mount points

sudo file -s /dev/xvdf # check if the device has a filesystem

sudo mkfs -t ext4 /dev/xvdf # format as ext4 (new volumes only!)

sudo mkfs -t xfs /dev/xvdf # format as xfs (new volumes only!)

sudo mkdir /data # create mount directory

sudo mount /dev/xvdf /data # mount the volume

df -h # verify size and mount point

sudo blkid /dev/xvdf # get UUID for fstab

sudo umount /data # unmount before detaching

sudo growpart /dev/xvdf 1 # extend partition after resize

sudo resize2fs /dev/xvdf # extend ext4 filesystem after resize

sudo xfs_growfs /data # extend xfs filesystem after resize
```

Pre-Launch Checklist

- ☐ Volume type matches the workload? (gp2 for general use, io1 for production databases)
- ☐ Availability Zone matches the target EC2 instance?
- ☐ Encryption enabled on the volume?
- ☐ Volume tagged with Name, Environment, and Owner?
- ☐ Volume pre-warmed? (io1 volumes only — write zeros before first production use)
- ☐ Snapshot schedule automated? (AWS Data Lifecycle Manager or AWS Backup)
- ☐ Cross-region snapshot copy configured for disaster recovery?

- [] /etc/fstab updated for auto-mount on reboot? (use UUID + nofail option)
- [] CloudWatch alarms set for IOPS, throughput, and queue depth?
- [] File system extended after any volume resize? (growpart + resize2fs or xfs_growfs)

Reference: aws.amazon.com/ebs | docs.aws.amazon.com/AWSEC2/latest/UserGuide/AmazonEBS.html